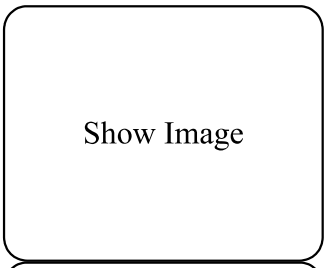
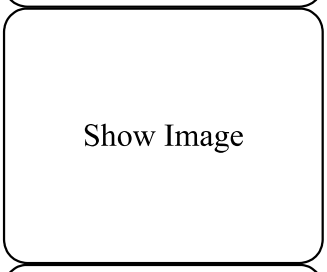
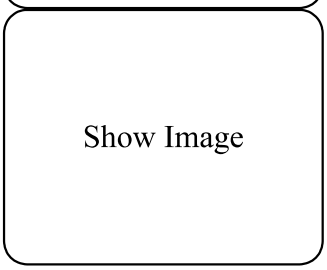
🎙️ Deepfake Speech Detection System

A comprehensive deep learning system for detecting synthetic/deepfake speech using an ensemble of three complementary models with SHAP explainability.







📋 Table of Contents

🎯 Overview

This project implements a state-of-the-art deepfake speech detection system that combines three different deep learning approaches:

1. **Wav2Vec 2.0**: Transformer-based audio representation learning
2. **Mel-Spectrogram CNN**: Convolutional neural network on mel-spectrograms
3. **MFCC Model**: CNN on mel-frequency cepstral coefficients

The system uses an ensemble voting mechanism to combine predictions from all three models, providing robust detection of synthetic speech.

✨ Features

- **Multi-Model Ensemble**: Combines three complementary deep learning models
- **MUSAN Augmentation**: Optional noise augmentation for improved robustness
- **SHAP Explainability**: Transparent model decisions using SHAP values
- **User-Friendly Interface**: Gradio-based web UI for easy interaction
- **Batch Processing**: Analyze multiple audio files simultaneously
- **Comprehensive Metrics**: Accuracy, precision, recall, F1-score, and confusion matrix
- **Flexible Configuration**: Easy-to-modify settings via config file

📐 Architecture

Model Details

1. Wav2Vec 2.0 Model

- Pre-trained on 960 hours of unlabeled speech
- Fine-tuned for binary classification (real/fake)
- Attention-based pooling for temporal aggregation
- Best for capturing high-level semantic features

2. Mel-Spectrogram CNN

- Processes 128-band mel-spectrograms
- 5-layer convolutional architecture
- Global average pooling
- Best for detecting frequency-domain artifacts

3. MFCC Model

- 40 MFCC coefficients with delta and delta-delta
- 4-layer CNN architecture
- Optimized for prosodic features
- Best for articulation and timing patterns

Ensemble Strategy

The system supports three voting strategies:

- **Majority Voting**: Simple majority wins
- **Weighted Voting**: Models weighted by validation performance
- **Average Voting**: Average of probability distributions

🚀 Installation

Prerequisites

- Python 3.8 or higher
- CUDA-capable GPU (recommended)
- 16GB RAM minimum
- 50GB free disk space (for dataset and models)

Step 1: Clone the Repository

```bash
git clone https://github.com/yourusername/deepfake-speech-detection.git
cd deepfake-speech-detection
```

Step 2: Create Virtual Environment

```bash
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
```

Step 3: Install Dependencies

```bash
```

```
pip install -r requirements.txt
```

Step 4: Download Dataset

Option A: Automatic Download (requires Kaggle API)

1. Create a Kaggle account at https://www.kaggle.com
2. Go to Account → API → Create New API Token
3. Place `kaggle.json` in `~/.kaggle/`
4. Run the download script:

```
bash

python utils/download_data.py
```

Option B: Manual Download

1. Download the FoR dataset from: https://www.kaggle.com/datasets/mohammedabdeldayem/the-fake-or-real-dataset
2. Extract to `data/raw/`
3. Optionally download MUSAN from: http://www.openslr.org/17/
4. Extract to `data/musan/`

📖 Usage

Quick Start

1. **Train Models**:

```
bash

python training/train_models.py
```

2. **Launch Web Interface**:

```
bash

python ui/gradio_app.py
```

3. Open your browser to `http://localhost:7860`

Command Line Prediction

```python
from inference.ensemble_predictor import EnsemblePredictor

# Initialize predictor
predictor = EnsemblePredictor()

# Make prediction
result = predictor.predict('path/to/audio.wav')

# Print results
print(f"Prediction: {result['ensemble']['label']}")
print(f"Confidence: {result['ensemble']['confidence']:.2%}")
print(f"Best Model: {result['best_model']['name']}")
```

Batch Processing

```python
# Batch prediction
audio_files = ['audio1.wav', 'audio2.wav', 'audio3.wav']
results = predictor.batch_predict(audio_files)

for result in results:
    print(f"{result['file']}: {result['ensemble']['label']}")
```

🎓 Model Training

Training from Scratch

```bash
# Train all three models
python training/train_models.py
```

Training Individual Models

```python
```

```python
from training.train_models import ModelTrainer
from models.wav2vec_model import create_wav2vec_model
from preprocessing.dataset import create_data_loaders, load_and_split_data

# Load data
data_split = load_and_split_data('data/raw')

# Create data loaders
train_loader, val_loader, test_loader = create_data_loaders(
    data_split['train_files'], data_split['train_labels'],
    data_split['val_files'], data_split['val_labels'],
    data_split['test_files'], data_split['test_labels'],
    feature_type='wav2vec'
)

# Create and train model
model = create_wav2vec_model(pretrained=True)
trainer = ModelTrainer(model, 'wav2vec')
history = trainer.train(train_loader, val_loader)
```

Training Configuration

Edit `config.py` to adjust training parameters:

```python
BATCH_SIZE = 16
LEARNING_RATE = 1e-4
NUM_EPOCHS = 20
EARLY_STOPPING_PATIENCE = 5
```

🎵 MUSAN Augmentation

What is MUSAN?

MUSAN (Music, Speech, and Noise) is a corpus of noise recordings used for data augmentation. Adding background noise during training helps models become robust to real-world acoustic conditions.

When to Use MUSAN?

Use MUSAN if:

- Your test data may contain background noise
- You want maximum robustness

- You have sufficient computational resources

Skip MUSAN if:

- Your test data is clean studio recordings
- You have limited disk space (<10GB)
- Training time is critical

Configuration

```python
# In config.py
USE_MUSAN_AUGMENTATION = True
AUGMENTATION_PROBABILITY = 0.3  # Apply to 30% of samples
NOISE_SNR_RANGE = (5, 15)  # Signal-to-noise ratio range
```

Assessment

Run the assessment script:

```bash
python preprocessing/audio_preprocessing.py
```

This will print a detailed analysis of whether MUSAN augmentation is recommended for your use case.

🔍 SHAP Explainability

What is SHAP?

SHAP (SHapley Additive exPlanations) provides model interpretability by showing which features contributed most to each prediction.

Generate Explanations

```python
```

```python
from inference.shap_explainer import SHAPExplainer
from models.wav2vec_model import create_wav2vec_model

# Load model
model = create_wav2vec_model()
# ... load trained weights ...

# Create explainer
explainer = SHAPExplainer(model, 'wav2vec')

# Generate explanation
explanation = explainer.explain_prediction(
    'audio.wav',
    save_path='explanation.png'
)
```

Global Feature Importance

```python
# Analyze feature importance across multiple samples
shap_values = explainer.generate_global_explanation(
    data_dir='data/raw',
    num_samples=100,
    save_path='global_importance.png'
)
```

📊 Results

Expected Performance

On the FoR dataset, you can expect:

Model	Accuracy	Precision	Recall	F1-Score
Wav2Vec	94-96%	0.93-0.95	0.94-0.96	0.94-0.95
Mel-CNN	92-94%	0.91-0.93	0.92-0.94	0.92-0.93
MFCC	90-92%	0.89-0.91	0.90-0.92	0.90-0.91
Ensemble	**96-98%**	**0.95-0.97**	**0.96-0.98**	**0.96-0.97**

Viewing Results

After training, results are saved to `results/`:

- `training_results.json`: Detailed metrics for each model
- `training_history.png`: Loss and accuracy curves
- `logs/`: TensorBoard logs for each model

View TensorBoard logs:

```bash
tensorboard --logdir results/logs
```

📁 Project Structure

```
deepfake_speech_detection/
├── config.py              # Configuration settings
├── requirements.txt       # Python dependencies
├── README.md              # This file
│
├── data/                  # Data directory
│   ├── raw/               # Raw audio files
│   ├── processed/         # Preprocessed data
│   └── musan/             # MUSAN noise corpus
│
├── models/                # Model architectures
│   ├── wav2vec_model.py   # Wav2Vec implementation
│   ├── mel_cnn_model.py   # Mel-spectrogram CNN
│   └── mfcc_model.py      # MFCC model
│
├── preprocessing/         # Data preprocessing
│   ├── audio_preprocessing.py  # Audio utilities
│   └── dataset.py         # PyTorch datasets
│
├── training/              # Training scripts
│   └── train_models.py    # Main training script
│
├── inference/             # Inference and prediction
│   ├── ensemble_predictor.py  # Ensemble prediction
│   └── shap_explainer.py  # SHAP explainability
│
├── ui/                    # User interface
```

```
│   └── gradio_app.py      # Gradio web app
│
├── utils/                 # Utility scripts
│   └── download_data.py   # Data download script
│
└── results/               # Training results
    ├── logs/              # TensorBoard logs
    └── temp_shap/         # Temporary SHAP plots
```

⚙️ Configuration

Key settings in `config.py`:

Paths

```python
DATA_DIR = 'data/'
MODELS_DIR = 'models/'
RESULTS_DIR = 'results/'
```

Audio Processing

```python
SAMPLE_RATE = 16000
MAX_AUDIO_LENGTH = 10  # seconds
MIN_AUDIO_LENGTH = 1
```

Model Settings

```python
WAV2VEC_MODEL = 'facebook/wav2vec2-base'
BATCH_SIZE = 16
LEARNING_RATE = 1e-4
NUM_EPOCHS = 20
```

Ensemble

```python
```

```
VOTING_STRATEGY = 'weighted'  # 'majority', 'weighted', 'average'
MODEL_WEIGHTS = {
    'wav2vec': 0.4,
    'mel_cnn': 0.3,
    'mfcc': 0.3
}
```

🔧 Troubleshooting

Common Issues

1. CUDA Out of Memory

Solution: Reduce batch size in `config.py`:

```
python

BATCH_SIZE = 8  # or even smaller
```

2. Models Not Loading

Solution: Ensure models are trained and saved:

```
bash

ls models/
# Should see: wav2vec_best.pth, mel_cnn_best.pth, mfcc_best.pth
```

3. Audio File Not Loading

Solution: Convert to WAV format:

```
bash

ffmpeg -i input.mp3 -ar 16000 output.wav
```

4. Slow SHAP Generation

Solution: Reduce background samples in `config.py`:

```
python

SHAP_BACKGROUND_SAMPLES = 20  # default is 50
```

Performance Optimization

GPU Utilization

```bash
# Monitor GPU usage
nvidia-smi -l 1
```

CPU Parallelization

```python
# In dataset.py
DataLoader(..., num_workers=4)  # Adjust based on CPU cores
```

🤝 Contributing

Contributions are welcome! Please follow these steps:

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/amazing-feature`)
3. Commit your changes (`git commit -m 'Add amazing feature'`)
4. Push to the branch (`git push origin feature/amazing-feature`)
5. Open a Pull Request

Development Setup

```bash
# Install development dependencies
pip install -r requirements-dev.txt

# Run tests
pytest tests/

# Check code style
flake8 .
black .
```

📄 License

🙏 Acknowledgments

- **FoR Dataset**: Mohammed Abdeldayem for the Fake-or-Real dataset
- **MUSAN**: David Snyder et al. for the MUSAN corpus
- **Wav2Vec 2.0**: Facebook AI Research
- **SHAP**: Scott Lundberg for SHAP library
- **Gradio**: Gradio team for the excellent UI framework

📧 Contact

For questions or support, please:

- Open an issue on GitHub
- Email: your.email@example.com
- Twitter: @yourusername

📚 Citations

If you use this project in your research, please cite:

```bibtex
@software{deepfake_speech_detection,
  title={Deepfake Speech Detection System},
  author={Your Name},
  year={2024},
  url={https://github.com/yourusername/deepfake-speech-detection}
}
```

Built with ❤️ for audio security